QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control 3
PRELIMINARY ROUND (Test Strategy & Planning) 3
1. Scope & Requirements Traceability 3
1.1 In-Scope Core Features 3
1.2 Out-of-Scope 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 4
4.1 Integration Thresholds (Merging to Main) 6
5. Test Case Specifications (Drafts) 7
6. AI Output & Boundary Testing (Drafts) 8

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	MediFlow - AI Hospital Triage & Appointment System
Team Repo URL	https://github.com/enxxxi/MediFlow.git
Project Board URL	https://trello.com/b/OlwC2GfN/mediflow-ai-hospital-triage-appointment-system
Live Deployment URL (optional for preliminary round)	Backend: https://mediflow-526a.onrender.com Frontend: https://mediflow-delta.vercel.app/

Objective:

The primary objective of **MediFlow** is to automate manual healthcare triage workflows by utilizing the mandatory **Z AI (GLM) Model** to orchestrate multi-agent coordination between patient symptom analysis and doctor appointment scheduling. The system aims to ensure 100% accuracy in identifying critical emergency cases while maintaining a system response time of less than 800ms to ensure real-world technical feasibility.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section ensures that every test aligns with the core requirements of an automated medical workflow.

1.1 In-Scope Core Features

- **AI Triage Engine:** describe symptoms, analyze symptoms, display result, give suggestions
- **Appointment Management:** book appointments, view available doctor and slot, view appointments
- **Emergency mode:** get immediate assistance, find nearby hospitals

1.2 Out-of-Scope

- Payment processing for medical bills
- Patient profile and medical record
- Customize app themes

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

Medical systems require strict risk anticipation to ensure no critical failure happens that endangers lives. The risk of the app system is being measured with the Risk Score formula, so Risk Score = Likelihood (L) x Severity (S).

Technical Risk	**Likelihood (1–5)**	**Severity (1–5)**	**Risk Score (L×S)**	**Mitigation Strategy**	**Testing Approach**
AI misdiagnosis	2	5	10 (Med)	Implement a "red-flag" keyword filter to bypass AI for immediate human alert.	Conduct testing with ambiguous but life-threatening symptom descriptions.
System timeout	3	4	12 (High)	Set a hard 800ms API response threshold.	Load testing with 50+ concurrent "Start" requests via Postman.

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	**Risk Level**	**Recommended Action**
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required

© UMHackathon 2026

Email: umhackathon@um.edu.my

16 – 25	Critical	Priority is Highest. Need extensive level of testing.

3. Test Environment & Execution Strategy

The testing environment for **MediFlow** is designed to validate the reliability of AI-driven medical triage workflows in a controlled, multi-tier architecture before deployment.

- **Unit Test**
  - Scope: AI triage logic and symptom parsing
  - Execution: The tests are executed via **Vitest**; developers run tests locally using npm test and the CI pipeline triggers them on every push to the repository
  - Isolation: To focus only on frontend logic, backend Z AI Model (GLM) responses and data fetching are mocked using handlers
  - Pass condition: components must render correctly across different screen sizes and state conditions must occur without console errors.

- **Integration Test**
  - Scope: authentication (login/sign up) to triage and appointment booking service
  - Workflow: verification of protected routes ensuring that unauthorized uses are redirected to login page
  - Pass condition: integration is successful when the AppProvider correctly propagates state from the AI triage engine to the appointment scheduling page

- **Test Environment (CI/CD Practice):**
  - Local testing: conducted in a Vite development environment using npm run dev
  - Staging/CI: GitHub Actions performs production build to ensure there are no TypeScript transpilation errors
  - Automation gate: merging to the main branch requires passing Linting and a 100% build success rate

- **Regression Testing & Pass/Fail Rules:**
  - Execution phase: full-scale regression testing is triggered automatically via Github each time a feature branch is merged into the main branch
  - Pass/Fail condition: the test case pass is only considered when the UI state, API response from Z AI model and dataset perfectly align with the expected behavior. Any mismatch is marked as Failed and logged in the Defect Log
  - Continuation Rule: follow the Agile principle of iterative development, basic smoke test must meet a 100% pass threshold before the CI/CD pipeline proceeds

- **Test Data Strategy:**
  - Automated: We utilize JSON-based seed files containing various symptom descriptions to test the Z AI Model (GLM) classification accuracy
  - Data privacy: all test data is synthetic and contains no sensitive personal

© UMHackathon 2026

Email: umhackathon@um.edu.my

information

- **Passing Rate Threshold:**
    - General threshold: a minimum of 85% of all total test cases that are executed must pass
    - Critical threshold: 100% pass rate is mandatory for all critical tests, including AI triage accuracy and security gates for protected routes

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	**Requirements**	**Project**	**Pass/Failed**
Build success	**Zero transpilation error**	**0 Errors**	**Passed**
Unit Tests	**100% Passing Rate**	**100%**	**Passed**
Code Quality	**Zero ESLint Warnings**	**100%**	**Passed**
AI Output	**Minimum 80% accuracy**	**85%**	**Passed**

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	**Minimum 90%**	**91%**	**Passed**
AI Output Pass Rate	**Minimum of 80% for Z AI Model triage classification**	**85%**	**Passed**
Critical Bugs	**Zero P0/01 Bugs**	**0**	**Passed**
API Performance	**Response Time < 800ms**	**37.69s**	**Failed**
Security	**API keys are not exposed**	**Clean**	**Passed**

5. Test Case Specifications (Drafts)

This section outlines the critical validation scenarios for the MediFlow MVP.

- **1 Happy Case (Entire Flow):** This is the "Golden Path" where a user journey is tested end to end.
- **1 Specific Case (Negative/Edge Test):** This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.
- **2 Non-Functional (NFR) Tests (Performance/Load):** This test consists of non-feature-based tests but architectural tests.

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case:** Full triage flow	Verify whether a user can log in, describe symptoms, receive AI triage result, and book appointment	1. Open Website, login to MediFlow 2. Navigate to triage and enter symptoms for AI triage 3. Review	Users should be able to login successfully. After entering symptoms and clicking the "submit" button, the result is	Users are able to login. Triage classification is accurate. Navigation is seamless. Status: Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			triage result 4. Select follow up action (book appointment, activate emergency mode)	displayed and follow up questions are asked for more accurate triage. Follow up actions are provided according to the triage level.	
TC-02	**Specific Case (Negative):** Unauthorized access	Verify the system blocks unauthenticated access to protected medical data without triggering application crashes.	1. Logout from MediFlow 2. Manually modify the URL to enter the appointments page 3. Observe system behaviour	The ProtectedRoute component identifies the missing token. The user is immediately redirected to the login page.	404 error occurred. Status: Failed
TC-03	**NFR (Performance):** Agentic workflow latency	Verify that the **GLM Orchestration** backend remains responsive under moderate concurrent load	1. Open Postman/JMeter 2. Send 50 concurrent POST requests to the triage endpoint 3. Monitor response times	Average API response time must remain < 800ms	Average API response time = 9ms Status: Passed

6. AI Output & Boundary Testing (Drafts)

This section validates that the Z AI (GLM) integration produces clinically relevant output and handles abnormal inputs gracefully.

6.1. **Prompt/Response Test Pairs**

© UMHackathon 2026

Email: umhackathon@um.edu.my

The following prompt pairs are used to test the MediFlow triage engine.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	“I've had a mild cough for 3 days, no fever, no shortness of breath.”	Output should identify “emergency” status, and provide the user option to activate emergency mode	EMERGENCY_ REDIRECTE D; emergency action page; no normal appointment scheduling	Pass
AI-02	“I have fever and headache since yesterday.”	Output should identify “non-urgent” status, and provide the user option to book appointment	Appointment scheduled; Department: General; Status: confirmed	Pass
AI-03	“I have vomiting and dizziness for 6 hours”	Output should identify “urgent” status, and provide the user option to book appointment	Urgent/high-prio rity appointment; reason: vomiting, dizziness	Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	2000 characters / 400 words
Input used while testing	100 000 characters

© UMHackathon 2026

Email: umhackathon@um.edu.my

Expected Behavior	System should either reject the input via 413 Payload Too Large or process it with significant latency
Actual Behavior	201 Created: the backend successfully received the 100k payload. However, latency increased to 51.22s, exceeding the 800ms performance threshold
Status	Pass(Resilience) / Fail (Performance)

6.3. Adversarial/Edge Prompt Test

This section documents how GLM responds to adversarial prompts and how MediFlow is going to handle it.

Fields	Details
Prompt used	“My chest is pain but i think it’s ok”
Expected behaviour	The system prioritizes the life-threatening symptom and trigger emergency mode
Actual behaviour	The AI model fail to recognize dreadful event and misclassify as “non-urgent”
Handling strategy	Use keyword-triggered redirection and strict system prompting

6.4. Hallucinating Handling

This section explain how MediFlow handle hallucination (e.g., prompt that is not related to medical symptoms)

Fields	Details
Prompt used	“I’m boring”
Expected behaviour	The AI model responds that the input is not medical symptom and prompt user to re-enter symptom or forced into a triage state

© UMHackathon 2026

Email: umhackathon@um.edu.my

Actual behaviour	The AI model attempted to map non-medical text into the closest available medical category and misclassify non-medical data as “injuries”
Mitigation strategy	We are implementing a Semantic Guardrail layer. Before the triage orchestration begins, a "Relevance Check" agent will verify if the input is medical. If the score is low, the workflow will terminate with an "Invalid Input" message rather than proceeding to misclassification.

© UMHackathon 2026

Email: umhackathon@um.edu.my